



PDF Download
2578128.2578238.pdf
15 February 2026
Total Citations: 5
Total Downloads: 249

 Latest updates: <https://dl.acm.org/doi/10.1145/2578128.2578238>

RESEARCH-ARTICLE

Variability viewpoint to describe reference architectures

MILENA GUESSI, University of São Paulo, Sao Paulo, SP, Brazil

FLAVIO OQUENDO, University of South Brittany Campus of Vannes, Vannes, Brittany, France

ELISA YUMI NAKAGAWA, University of São Paulo, Sao Paulo, SP, Brazil

Open Access Support provided by:

University of São Paulo

University of South Brittany Campus of Vannes

Published: 07 April 2014

[Citation in BibTeX format](#)

WICSA '14: Proceedings of the WICSA
2014 Companion Volume
April 7, 2014
Sydney, Australia

Variability Viewpoint to Describe Reference Architectures

Milena Guessi
Dept. of Computer Systems
University of São Paulo - USP
PO Box 668, 13560-970
São Carlos, Brazil
milena@icmc.usp.br

Flavio Oquendo
IRISA Research Institute
University of South Brittany
PO Box 573, 56017
Vannes, France
flavio.oquendo@univ-ubs.fr

Elisa Yumi Nakagawa
Dept. of Computer Systems
University of São Paulo - USP
PO Box 668, 13560-970
São Carlos, Brazil
elisa@icmc.usp.br

ABSTRACT

Reference architectures have emerged as a special type of software architecture that achieves well-recognized understanding of specific domains. Their purpose is therefore to be a guidance for the development, standardization, and evolution of systems of such domains or neighbor domains. Adequate representation of such architectures is essential to promote their effective use and dissemination, using, for instance, different architectural viewpoints. A comprehensive description of reference architectures should not only record common features and functionalities, but also variations that could be present in the instances of these architectures. In this scenario, the main contribution of our work is to propose an architecture viewpoint to represent variability in reference architectures. We also describe the steps for creating such viewpoint and present an example of a technique that could be used to represent it. A case study is also presented, demonstrating the feasibility of our approach. Based on initial results, we have observed that the variability viewpoint could contribute to a more adequate, complete description of reference architectures and, as a consequence, it could promote a more effective dissemination and use of such architectures.

Categories and Subject Descriptors

D.2.11 [Software Engineering]: Software Architectures—*data abstraction, domain-specific architectures*

General Terms

Theory

Keywords

Architecture Description, Reference Architecture, Variability

1. INTRODUCTION

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.
WICSA '14, April 07 – 11 2014, Sydney, NSW, Australia
<http://dx.doi.org/10.1145/2578128.2578238>
Copyright 20XX ACM 978-1-4503-2523-3/14/04 ...\$15.00.

Software architectures constitute the backbone for any successful software-intensive system and play a fundamental role in determining the system quality (e.g., performance, portability, and maintainability) [14, 26, 28]. Recently, we have also observed the increasing concern in treating variability in the context of software architectures, as a response to an even greater necessity of developing similar systems. Variability has been investigated in several research topics, such as, self-adaptive systems, configurable interfaces, software-product line, and service-based systems with dynamic runtime composition [10]. For instance, product line architectures explicitly address commonality and variability as basis for all product line members. In this scenario, variability can be understood as the software artifact ability to be tailored for a specific context in a preplanned manner [10]. It is quite important to say that software architectures constitute the main element to promote variability [10].

In the context of software architectures, reference architectures appear as a special type of software architecture that captures the essence of the architectures of a collection of systems in a given domain. The purpose of a reference architecture is to provide guidance for developing, standardizing, and evolving software architectures of a specific domain. These architectures establish business rules, architectural styles (sometimes also defined as architectural patterns that address quality attributes in the reference architecture), best practices of software development (for instance, architectural decisions, domain constraints, legislation, and standards), and the software elements that support development of systems for that domain. All of these must be supported by a unified, unambiguous, and widely understood domain terminology [20]. Given their relevance for the construction of software systems, several reference architectures can be found in the literature, such as for Ambient Assisted Living (AAL) [6, 27] and for automotive domain [3].

The design of reference architectures has been mainly focused on identifying commonalities among systems of a given domain and then framing this information into their design [2, 5, 7, 9, 21]. However, considering that reference architectures are basis of a set of software systems that sometimes differ in some points, it is also interesting to investigate how variability could be treated in reference architectures [9, 19]. For example, Ortiz et al. [25] present a reference architecture for teleoperated service robots, which considers the high variability of functionalities, devices, and physical characteristics. Weyns [29] presents a reference architecture for situated multiagent systems in which variability is explicitly described using formal specifications. In this perspec-

tive, describing variability in reference architectures is also important to make these architectures into comprehensive bodies of knowledge of specific domains. Moreover, variability can also transform the use of reference architectures into a more straightforward process, since variations will be already foreseen in these architectures. Nevertheless, in spite of the idea of treating variability in such architectures, there is still no solid understanding on how variability should be properly addressed in reference architectures [19].

The main contribution of this study is to propose a particular architecture viewpoint, named variability viewpoint, to support the description of variability in reference architectures. We also present a case study to evaluate our approach. The domain of embedded systems is used to illustrate our proposal. Preliminary results have pointed out that the use of this viewpoint, together with other architectural viewpoints, makes it possible to have a more comprehensive architectural description of reference architectures, and leverages the possibilities of disseminating and using these architectures.

This paper is organized as follows. In Section 2 we present a brief background on architectural descriptions of both software architectures and reference architectures. In Section 3 we present the variability viewpoint and a technique to represent it. In Section 4 we describe the case study where we captured users impression on this viewpoint. Finally, in Section 5 we summarize our contributions and discuss perspectives of future work.

2. BACKGROUND

Software architecture description is the main set of the artifacts that makes it possible to represent the design of software architectures. Besides playing a major role in the communication among stakeholders, the architecture description can also be used in the analysis of architectural trade-offs, as well as documentation and as input for automatic simulation tools [11]. Motivated by their relevance in the software architecture life cycle, the architecture description has been the focus of important studies [4, 12]. In particular, ISO/IEC/IEEE 42010 standard [11], also known as IEEE 1471-2000, represents an important effort towards describing software architectures. As proposed by this standard, the usage of multiple architectural viewpoints is a common practice to document architectures of software systems. An architectural viewpoint frames particular concerns of the system stakeholders and consists of the conventions for the construction, interpretation, and use of an architectural view [11]. An architectural view, consisting of one or more architectural models, is a representation of a set of system elements and relations associated with them, conforming to a specific viewpoint [11]. Several viewpoints, such as logical, runtime, data, and physical viewpoints, have been proposed to represent software architectures as in [13].

In order to represent such models, views, and viewpoints, Architecture Description Languages (ADLs) can be used. Given the diversity of ADLs, it is useful to classify them into formal and semi-formal languages. Formal languages, e.g., Wright [1] and Rapide [16], use formal syntax and semantics, while semi-formal languages, e.g., UML [24] and SysML [23], are usually based in graphical notations. The main advantages of using formal languages are verification & validation support, automatic generation of models and code, and an accurate description of the system behavior

[17]. However, since formal languages are usually difficult for non-technical stakeholders to use and to understand, semi-formal languages and even informal representations have been preferred. It is important to highlight that semi-formal languages increase understandability, at the cost of lowering formal accuracy.

In parallel, for reference architectures, an essential activity for their success is their adequate representation (or architectural description), making it possible to make them understandable for a wide variety of stakeholders (such as customers, product managers, project managers, and engineers). In this perspective, in spite of most of these architectures are represented in an informal way (using box-and-lines, for instance), initiatives that propose means to more adequately document them can be identified. In general, architectural viewpoints and modeling techniques are adopted. Lopez et al. [15] proposed the use of the Enterprise Viewpoint, Computational Viewpoint, and Information Viewpoint. Meland et al. [18] suggested the use of the Component and Connector Viewpoint and Information Model. Nakagawa and Maldonado [22] proposed the use of the Deployment Viewpoint, Runtime Viewpoint, and Module Viewpoint. To represent these viewpoints, different UML (Unified Modeling Language) diagrams, such as Use Case Diagram, Component Diagram, Class Diagram, and Deployment Diagram, were adopted [15, 18, 22]. These initiatives already brought important contributions to the representation of reference architectures. However, there is still a lack of studies that investigate how to completely represent them, including discussions about variability in reference architectures. This scenario has therefore motivated the conduction of this work.

3. THE VARIABILITY VIEWPOINT

Aiming at capturing the variability in reference architectures in an organized manner, we propose a specific viewpoint, named variability viewpoint. This viewpoint presents the elements of a reference architecture that are: (i) optional; (ii) variable (i.e., they change depending on the software system to be developed); and (iii) all possible alternatives (i.e., they can be found in different software systems derived from this architecture). It is worth highlighting that the variability viewpoint can be considered a crosscutting viewpoint. In another study [21], we have proposed the crosscutting viewpoint to show general information about the reference architecture, such as terms/concepts and variabilities. The information described in the crosscutting viewpoint is transversal to other viewpoints and serves as basis to the construction of other views. For instance, a variability view can represent a specific variable/optional element and its variants that are related to elements of the structural view and deployment view at the same time.

In this initial study, we experimented the use of SysML (Systems Modeling Language) in order to observe its feasibility to represent variability of reference architectures, in particular, of the embedded systems domain. SysML is an OMG standard that extends the UML to fill in the semantic gap between systems, software, and other engineering disciplines [23]. The semantics of SysML enables the creation of an integrated model — i.e., model elements in one diagram can be related to model elements in other diagram. As a result, the variability viewpoint is able to crosscut all other viewpoints in the reference architecture description. Two

particular elements of SysML are used in this viewpoint: (i) blocks that define system structure and properties; and (ii) ports that support the specification of information, matter, and energy exchange between system elements.

Specifically, we selected the Internal Block Diagram of SysML to represent the variability viewpoint. This diagram enables representation of the internal structure of a higher level block. Thus, it is possible to represent associations and interfaces between parts of a block, whereas a part (or usage) can be understood as a role [8]. Parts in the diagram are indicated with a colon (:). A part enables the same block to be used in different contexts and to be uniquely identified by its usage. Each part may have unique behaviors, properties, and constraints that apply to its particular usage. Ports specify interaction points with other parts and are represented by small squares on the boundary of the parts. Optionally, connectors can be used to define how the parts connect to one another via their ports and are represented as lines between ports. It is important to say that we have extended the notation of this diagram in order to highlight the common elements of a reference architecture. For this, we have proposed to use a thicker outline to represent them. Conversely, variable elements use the regular notation, i.e., a thinner outline. Figure 1 shows this notation. This figure depicts an optional block in which two ports are described: one of them is optional (noted with bold font) and the other one is variable. We also describe two parts for this block, where the first is optional (noted with thicker outline) and the second is variable (noted with thinner outline).

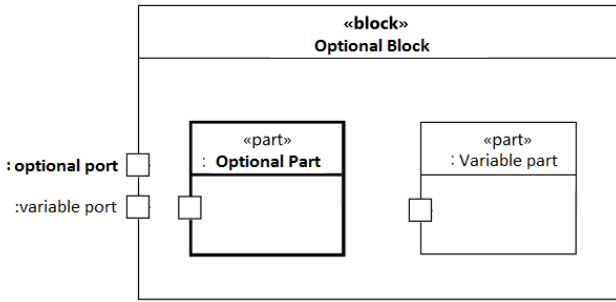


Figure 1: Notation to represent variability in SysML.

Regarding modeling tools to build the variability view, we can use, for instance, Papyrus¹ which supports this extended notation in the Internal Block Diagram.

In the next section, we present a case study using this notation and evaluating its feasibility as a diagram to represent variability in reference architectures.

4. CASE STUDY

We conducted a case study in order to evaluate our approach regarding its feasibility and usability. Our main objective was to identify how the variability viewpoint influenced the architecture description understandability. To do so, we created a new architecture description for Weyns (2006) [29] situated multiagent reference architecture. This reference architecture is composed by: one or more Agents, which are autonomous problem solving entities that can dy-

namically control their own behavior (i.e., perceive and actuate in the environment); an Application Environment, which is the medium for agents interaction, such as sharing information and coordinating behavior, that has to be designed for each concrete multiagent system; and an Deployment Context, comprised by the hardware, software, and other external resources with which the system interacts (e.g., sensors, actuators, printer, network, database, web service). In our architecture description for this reference architecture, we created a particular view to describe variability. In his original study, the author included throughout the architecture description textual explanations about variability points in the reference architecture and applicability conditions. Following our approach, all information about variability was centralized in a particular view, named variability view. Then, a diagram to illustrate structural variable points in the reference architecture was created. Based on the original architecture description, we also added to each diagram a textual explanation about the conditions under which the design variants could be applied.

The diagrams created for the variability view in our case study are presented in Figures 2 and 3. As similar to UML, abstract elements are represented in italic font. Figure 2 shows optional and variable structural elements in the reference architecture. The Deployment Context is not shown in Figure 2, because it is out of the scope of this reference architecture. In this Figure, it is possible to observe that the Agent is composed of four parts: Perception, Decision Making, Communication, and Current Knowledge. The Perception part can be requested by the Decision Making and Communication parts to provide a perception of the environment. The Decision Making part selects and realizes possible actions of the Agent in the environment. The Communication part determines how the Agent can exchange messages. The Current Knowledge works is a repository from which the Agent can store and retrieve its current state. According to our notation, based on Figure 2 it is possible to determine that the Communication part is variable among software systems. Then, there is a textual description next to this diagram explaining that the Communication part is not required in systems where the Agent does not communicate through message exchange. Figure 3 shows the main concepts of the reference architecture and highlights the concepts that differ among different software systems. For example, the Conversation concept refers to a set of messages exchanged among Agents. The content of these messages is determined by the Communication Language and the sequence in which they can be exchanged is determined by the Communication Protocol. According to our notation, based on Figure 3, it is possible to determine that the Communication Language and the Communication Protocol must be specified for a concrete software architecture.

In total, four subjects participated in our case study. The subjects were separated in two groups and each group analyzed a version of the architecture description. The original version of the architecture description is referred hereinafter to as “original”, while the architecture description we created using a particular view for describing variability is referred to as “new”. All subjects received a link to an on-line questionnaire where they answered a few questions about the reference architecture design. Regarding variability, we specifically inquired our subjects about which modules of the Application Environment could be omitted in instances

¹<http://www.eclipse.org/modeling/mdt/papyrus>

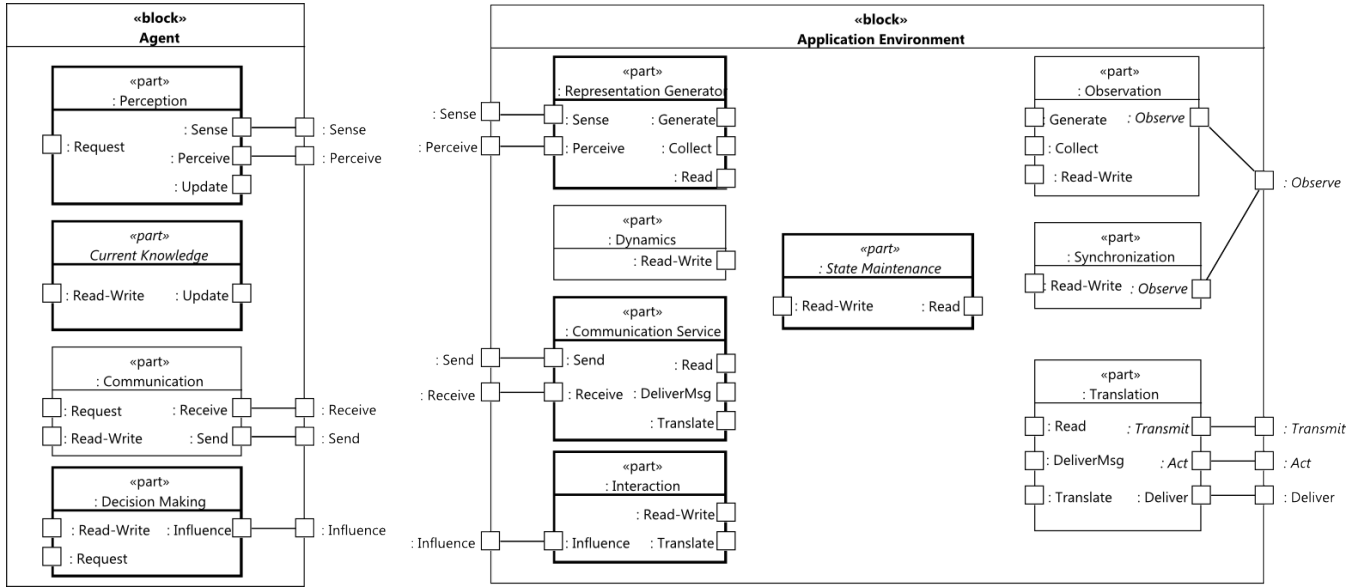


Figure 2: Main variability view for the situated multiagent system reference architecture

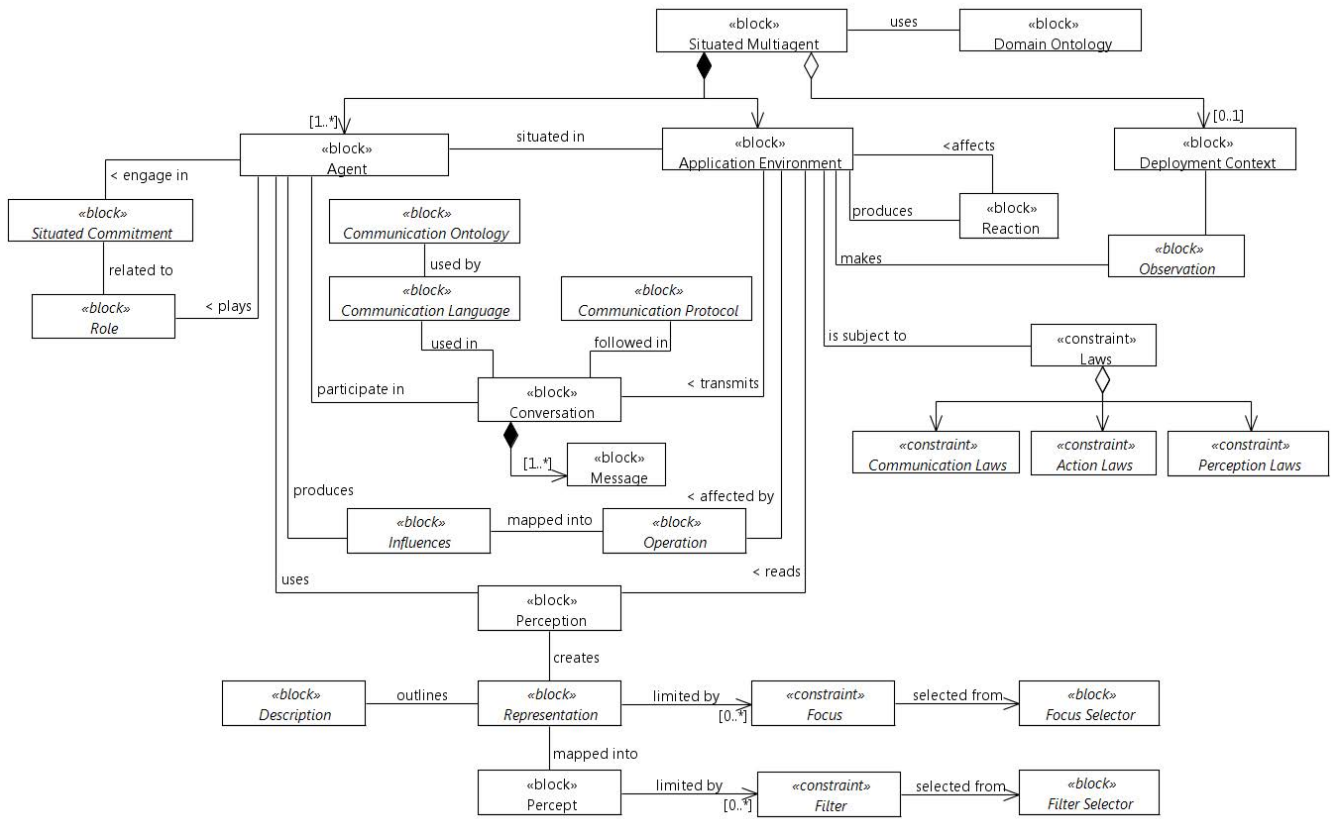


Figure 3: Concept variability view for the situated multiagent system reference architecture

of the reference architecture. We selected the modules (I) Translation, (II) Dynamism, and (III) Communication Service and defined five possible answers a) II; b) II, III; c) I, II; d) I, II, III; and e) Doesn't know. We added option (e) because we advised subjects against guessing. The correct answer was option (c). We also asked our subjects to grade from 1 (easy) to 4 (difficult) the difficulty level for answering this question. Table 1 shows the subjects answers and their informed difficulty level. From this initial result, it is possible to observe that both groups experienced some difficulty to answer this question, although there was a small advantage for the new architecture description.

Table 1: Subjects performance on the test

Version	New		Original	
Subject	#1	#2	#3	#4
Answer	Wrong	Correct	Correct	Wrong
Difficulty	3	2	4	2

Subjects that analyzed the new architecture description also informed their impressions regarding three aspects of the variability view: (i) usefulness, related to the contribution of the view to understand the reference architecture design; (ii) effort, related to the amount of time required to analyze or scan the view for a particular information; and (iii) clarity, related to the amount of time required to understand a particular information in the view. Subjects could grade each aspect from 1 (low) to 4 (high). Table 2 presents the subjects answers. We observe that the subjects perception on the usefulness of this view was very different.

Table 2: Subjects perception on the variability view

Version	New	Original
Subject	#1	#2
Usefulness	4	1
Effort	4	3
Clarity	1	2

To better understand the factors that could have played an important role in these results, we asked the subjects to complete a small questionnaire about their personal experience on the topics covered in the architectural description, such as UML, SysML, and Embedded Systems. Table 3 summarizes subject's background. We observe that all four subjects present solid knowledge on Software Engineering. Three of them are doctorate students and one of them is a post-doctorate student in Software Engineering. Apart from SysML, we also observe that both groups presented similar knowledge. The lack of a solid SysML background does not seem to have played a determinant role in the analysis of the new architectural description, since both groups presented similar performances. However, this factor must certainly have contributed to the effort and clarity perceptions of the variability view.

5. CONCLUSION

Reference architectures have become more and more important to software systems development, promoting mainly the reuse of knowledge contained in these architectures. In this context, their adequate representation through an architectural description is quite an important task. Recently,

Table 3: Subjects background on the covered topics in the architecture description

Version	New		Original	
Subject	#1	#2	#3	#4
UML	4	4	4	3
SysML	2	1	3	2
Embedded System	4	4	4	4
Software Architecture	4	2	3	3
Reference Architecture	4	2	3	3

the idea of treating variability in reference architectures has been proposed and variability has been already found as an important complement in the description of some of these architectures. In this scenario, the main contribution of this study was to propose an approach to represent such variability. In particular, we proposed a new architectural viewpoint – the variability viewpoint – to support representation of variability in reference architectures. We also evaluated our approach through a case study, using also a graphical notation that we have extended to represent this viewpoint. From this study, we have observed that this new viewpoint can enhance the clarity of reference architectures, since it can facilitate the process of scanning architectural descriptions. In the more perfect case, by using the variability viewpoint, in summary, instantiation of reference architectures (i.e., their use to create software architectures of specific systems) could be reduced to the selection of variants for the variable elements of such architectures.

As a future work, we intend to conduct more case studies and/or experiments in order to completely evaluate our approach. For this, we intend to consider reference architectures of different domains, such as ubiquitous systems and information systems, as well as stakeholders with different technical backgrounds. We also intend to propose adequate diagrams/notation for representing the variability viewpoint. With this, we intend to provide a means to achieve a more complete representation of reference architectures and, as a consequence, contribute to consolidate the area of reference architectures.

6. ACKNOWLEDGMENT

This work is supported by Brazilian funding agencies Fapesp (Grant N.: 2012/24290-5 and 2011/23316-8), Capes/Nuffic (Grant N.: 034/12), and CNPq (Grant N.: 474720/2011-0).

7. REFERENCES

- [1] R. J. Allen. *A Formal Approach to Software Architecture*. PhD thesis, Carnegie Mellon University, 1997.
- [2] S. Angelov, P. Grefen, and D. Greefhorst. A framework for analysis and design of software reference architectures. *Information and Software Technology*, 54:417–431, 2012.
- [3] AUTOSAR. AUTOSAR (AUTomotive Open System ARchitecture). [On-line], 2013. Available in <http://www.autosar.org/> (01/12/2014).
- [4] P. Clements, F. Bachmann, L. Bass, D. Garlan, J. Ivers, R. Little, P. Merson, R. Nord, and J. Stafford. *Documenting Software Architectures: Views and Beyond*. Addison-Wesley, 2 edition, 2011.

- [5] R. Cloutier, G. Muller, D. Verma, R. Nilchiani, E. Hole, and M. Bone. The Concept of Reference Architectures. *Systems Engineering*, 13(1):14–27, 2010.
- [6] Continua Health Alliance. Continua Health Alliance. [On-line], *World Wide Web*, 2013. Available in <http://www.continuaalliance.org/> (01/12/2014).
- [7] L. Dobrica and E. Niemela. An approach to reference architecture design for different domains of embedded systems. In *SERP 2008*, pages 287–293, Las Vegas, USA, 2008.
- [8] S. Friedenthal, A. Moore, and R. Steiner. *A Practical Guide to SysML: The Systems Modeling Language*. Morgan Kaufmann, 2008.
- [9] M. Galster, P. Avgeriou, D. Weyns, and T. Mannisto. Empirically-grounded reference architectures: A proposal. pages 153–157, Boulder, USA, 2011.
- [10] M. Galster, P. Avgeriou, D. Weyns, and T. Mannisto. Variability in software architecture: current practice and challenges. *SIGSOFT Software Engineering Notes*, 36(5):30–32, 2011.
- [11] ISO/IEC/IEEE 42010:2011. Systems and software Engineering — Architecture Description, 2011.
- [12] P. Kruchten. Architectural Blueprints - The “4+1” View Model of Software Architecture. *IEEE Software*, 12(6):42–50, 1995.
- [13] P. Kruchten. *The Rational Unified Process: An Introduction*. The Addison-Wesley Object Technology Series. 2 edition, 2000.
- [14] P. Kruchten, H. Obbink, and J. Stafford. The past, present, and future for software architecture. *IEEE Software*, 23(2):22–30, 2006.
- [15] D. M. Lopez and B. G. Blobel. A development framework for semantically interoperable health information systems. *Int. Journal of Medical Informatics*, 78(2):83–103, 2009.
- [16] D. C. Luckham. Rapide: A Language and Toolset for Simulation of Distributed Systems by Partial Orderings of Events. In *POMIV’1996*, volume 29 of *DIMACS*, pages 1–25, New Jersey, USA, 1996.
- [17] N. Medvidovic and R. N. Taylor. A Classification and Comparison Framework for Software Architecture Description Languages. *IEEE Transactions on Software Engineering*, 26(1):70–93, Janeiro 2000.
- [18] P. H. Meland, S. Ardi, J. Jensen, E. Rios, T. Sanchez, N. Shahmehri, and I. A. Tondel. An architectural foundation for security model sharing and reuse. In *ARES’09*, volume 1–2, pages 823–828, Fukuoka, Japan, 2009.
- [19] E. Y. Nakagawa. Reference Architectures and Variability: Current Status and Future Perspectives. In *WICSA/ECSA 2012*, pages 1–4, Helsinki, Finland, 2012.
- [20] E. Y. Nakagawa, P. O. Antonino, and M. Becker. Reference Architecture and Product Line Architecture: A Subtle But Critical Difference. In *ECSA 2011*, pages 207–211 (LNCS v. 6903), Essen, Germany, 2011.
- [21] E. Y. Nakagawa, M. Guessi, D. Feitosa, F. Oquendo, and J. C. Maldonado. Consolidating a process for the design, representation, and evaluation of reference architectures. In *WICSA’2014*, pages 1–10, Sydney, Australia, 2014.
- [22] E. Y. Nakagawa and J. C. Maldonado. Reference Architecture Knowledge Representation: An Experience. In *SHARK’2008 at ICSE’2008*, pages 51–54, Leipzig, Germany, 2008.
- [23] OMG. Systems Modeling Language v 1.2. [On-line], 2010. <http://www.omg.sysml.org/> (01/12/2014).
- [24] OMG. Unified Modeling Language v. 2.3. [On-line], 2010. <http://www.omg.org/spec/UML/2.3/> (01/12/2014).
- [25] F. J. Ortiz, J. A. Pastor, D. Alonso, F. Losilla, and E. de Jódar. A reference architecture for managing variability among teleoperated service robots. In *ICINCO 2005*, pages 322–328, Barcelona, Spain, 2005.
- [26] M. Shaw and P. Clements. The golden age of software architecture. 23(2):31–39, Mar/Apr 2006.
- [27] UniversAAL Project. The UniversAAL Reference Architecture. [On-line], *World Wide Web*, 2013. Available in <http://www.universaal.org/> (01/12/2014).
- [28] A. I. Wasserman. Towards a discipline of software engineering. *IEEE Software*, 13(6):23–31, 1996.
- [29] D. Weyns. *An Architecture-Centric Approach for Software Engineering with Situated Multiagent Systems*. PhD thesis, Katholieke Universiteit Leuven, Belgium, 2006.